

ERS: Energy-efficient Real-time DAG Scheduling on Uniform Multiprocessor Embedded Systems

Debabrata Senapati
SRM University - AP
Andhra Pradesh, India
debabratasenapati@gmail.com

Dharmendra Maurya
IIT Guwahati
Guwahati, India
mauryad335@gmail.com

Arnab Sarkar
IIT Kharagpur
Kharagpur, India
arnab@atdc.iitkgp.ac.in

Chandan Karfa
IIT Guwahati
Guwahati, India
ckarfa@iitg.ac.in

Abstract—Nowadays, many embedded systems such as mobiles and laptops to satellites and robotic systems, are often driven by limited energy sources like batteries. Hence, these devices are not only judged by their real-time and functional performance but also by their efficiencies in terms of energy management. Energy minimization is one of the primary design requirements for distributed embedded systems. The growing importance of complex applications in the distributed system introduces significant challenges in reducing energy consumption. This work addresses the problem of scheduling a real-time application abstracted as a directed acyclic graph (DAG), on a Dynamic Voltage and Frequency Scaling (DVFS) enabled uniform multiprocessor system, by proposing an efficient heuristic strategy called *Energy-efficient Real-time DAG Scheduler (ERS)*. *ERS* effectively selects the appropriate processing frequency for each task-to-processor pair in the system such that overall energy saving is maximized while satisfying constraints related to resource, task precedence and deadline. We have evaluated the performance of the proposed framework using real-world benchmark applications. Obtained results reveal that *ERS* is able to deliver better performance in terms of energy savings than state-of-the-art works such as *GSPM*, *SSPM*, and *PSPM*.

Index Terms—DAG Scheduling, List Scheduling, Energy-aware, Real-time, Uniform Multiprocessor Systems.

I. INTRODUCTION

Real-time embedded systems are computational systems that govern processes associated with the physical systems and must react to stimuli obtained from the physical systems within a strict time bound called deadline [1]. The accuracy of these systems depends not only on the delivered result but also on the completion time [2]. Some applications that demand real-time computing include pacemakers in health-care, anti-lock braking systems in vehicles, space missions, industrial automation, etc. These real-time applications are structured as Directed Acyclic Graphs (DAGs) (also alternatively referred to as *task graph* in this work), in which nodes denote execution units called tasks and directed edges represent inter-task dependencies [3]. *Given a distributed uniform multiprocessor platform and a real-time DAG-structured application, successfully allocating application tasks to the processors with start times while satisfying all timing, resources and precedence constraints, is eventually a scheduling problem.*

In general, DAG-scheduling problems are known to be NP-complete problems [4], [5]. Therefore, the researcher addresses issues in this domain using low-overhead heuristic solutions that deliver fast and acceptable schedules [6]. Traditionally, list

scheduling heuristics are known to deliver efficient schedules within reasonable time complexities [7], [8], [9], [10], [11]. The DAG scheduling heuristic problem is broadly classified into two types: (i) *static (offline)* and (ii) *dynamic (online)* [12]. In static schemes, the decisions, such as task-to-processor allocations and start times for task execution, are considered at design time. In contrast, dynamic schemes take such choices at run-time.

Energy efficiency has become a primary design constraint in battery-operated real-time embedded systems. Typically, two energy management schemes have been used: (i) *Dynamic Power Management (DPM)* [13], where selective parts of a system are turned off strategically at the idle time of the processors. (ii) *Dynamic Voltage and Frequency Scaling (DVFS)* [14] delivers energy-efficient scheduling by simultaneously scaling down the supply voltage and frequency of a processor. In this work, we attempt to solve the problem of *scheduling a real-time DAG application on distributed uniform multiprocessors, with the objective of minimizing dissipated energy.*

The rest of the article is organized as follows. Section II presents related works. Section III depicts system models and Section IV describes the proposed scheduling strategies. Section V discusses the simulation results and the article is summarized in Section VI.

II. RELATED WORKS

A comprehensive survey of various energy-aware scheduling techniques for real-time systems may be found in [15]. Weiser et al. in [16] first introduced an energy-saving policy by using fine-grain control of processor speed by an operating system scheduler. The primary goal is to monitor processor idle time to reduce energy consumption by lowering clock speed and idle time to a minimum. Subsequently, Yao et al. in [17] analyze offline and online policies to schedule tasks with arrival times and deadlines on a uniprocessor system with minimum energy consumption. These works have been extended further in [18], [19] to minimize processor energy consumption while still satisfying the deadline for task execution. Authors in [20] proposed energy-efficient scheduling using a dynamic voltage scaling technique for periodic real-time tasks. In [21], authors have minimized the energy consumption using the DVFS technique for independent periodic tasks on uniform multiprocessors. Authors in [22] solved the same problem using DVFS and DPM techniques. Authors in [13] solve

the energy minimization problem for real-time precedence-constrained applications on uniform multiprocessor systems. They first developed a static schedule to minimize the schedule length of the application. For real-time systems by utilizing the slack available between the deadline and application sink node (i.e., global slack), they proposed three algorithms, namely (1) Global Static Power Management (*GSPM*) (it uses the entire global slack on the first task of each processor), (2) Simple Static Power Management (*SSPM*) (it proportionally distribute the global slack to each task according to their execution times) and (3) Static Power Management with Parallelism (*PSPM*) (it distribute the slack based on task parallelism).

In this work, we first develop a list-based heuristic DAG scheduler called *ESUM* to minimize an application's *makespan* on uniform multiprocessor systems, which considers the message communication energy on the communication links. After that, the article develops a real-time scheduler called *ERS* to optimize energy consumption for uniform multiprocessors and compares *ERS* with the state-of-the-art algorithms, namely *GSPM*, *SSPM* and *PSPM*. Experimental evaluation shows that *ERS* is able to deliver better performance in terms of energy savings than state-of-the-art works.

III. SYSTEM MODELS

This section presents the application and platform model followed by the energy consumption model, related to the proposed work.

A. Application and platform model

The real-time application considered here is abstracted as Directed Acyclic Graph (DAG), $G(V, E)$, where V denotes a set of tasks $V = \{\tau_1, \tau_2, \dots, \tau_{|V|}\}$ and E represents a set of edges $e_{i,j} = (\tau_i, \tau_j) \in E$ which ensures the precedence constraints among the task pairs. Each edge $e_{i,j}$ is labeled with $data_{i,j}$ (i.e., $data_{1,2} = 18$; refer Fig. 1) that represents the size of the output message from task τ_i to τ_j . The task with no predecessors (successors) is called entry (τ_{entry} (exit τ_{exit})) task. The considered DAG is assumed to have a single entry and exit node. If an application has multiple entry (exit) tasks, then they are connected to a dummy entry (τ_{entry} (exit τ_{exit})) node using dummy edges. $pred(\tau_j)$ ($succ(\tau_j)$) denotes the set of all direct predecessors (successors) of task τ_j .

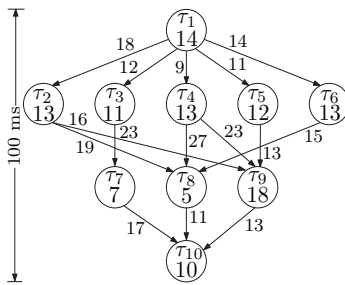


Fig. 1. An example DAG of 10 tasks.

The platform $U = \{u_1, u_2, \dots, u_{|U|}\}$ consists of $|U|$ uniform multiprocessors. The communications between any two

processors are assumed to perform without contention using a fully connected topology. An element $b_{x,y}$ of matrix B (size $|U| \times |U|$) is used to keep the data transfer rates between processors u_x and u_y . The message communication time ($c_{i,j}^{x,y}$) from τ_i (executes on u_x) to τ_j (on u_y) is: $c_{i,j}^{x,y} = \frac{data_{i,j}}{b_{x,y}}$, when τ_i and τ_j are scheduled on same processor then $c_{i,j}^{x,y} = 0$. Any processor say, $u_y \in U$ can operate at a set of $|F|$ discrete frequencies $F = \{f_{min}, f_{min+1}, \dots, f_{max}\}$, where f_{max} denotes the maximum available frequency. The Worst Case Execution Times (WCETs) of a task τ_j on any processor at its maximum frequency is ω_j (i.e., $\omega_1 = 14$; refer Fig. 1). Then, the WCET of τ_j on a processor at its operating frequency f_α can be calculated as:

$$\chi[\tau_j, f_\alpha] = \omega_j \times \frac{f_{max}}{f_\alpha} \quad (1)$$

B. Energy Consumption Model

The power consumption of a processor at frequency f_α is given by the expression [14], [21]:

$$P(f_\alpha) = P_s + h(P_{int} + P_d) = P_s + h(P_{int} + C_{ef} f_\alpha^m) \quad (2)$$

In equation 2, the static power P_s is consumed for the entire duration when the processor is *on*. The frequency-independent dynamic power P_{ind} can be removed by switching the system into sleep mode. The frequency-dependent dynamic power P_d is highly sensitive to the processor's operating frequency at a given time. The processor state is represented by h ($= 1$ or 0 denote *active* or *sleep* state). The effective switching capacitance C_{ef} and the dynamic power exponent m are processor-dependent constants.

Dynamic energy consumption for a task τ_j executing on a processor u_y (having maximum frequency f_{max}) at current frequency f_α is computed as:

$$E_{dp}(\tau_j, f_\alpha) = (P_{ind} + C_{ef} f_\alpha^m) \times \omega_j \times \frac{f_{max}}{f_\alpha} \quad (3)$$

Here, we consider the message communication energy is directly proportional to the message communication time. The communication energy consumption for a task τ_j can be determined as:

$$E_{com}(\tau_j) = \sum_{\tau_i \in pred(\tau_j)} ecr \times c_{i,j}^{x,y} \quad (4)$$

where ecr is the energy consumption rate per time unit of communication. From equations 3 and 4, the total energy consumed by a task τ_j can be expressed as:

$$E_d(\tau_j) = E_{dp}(\tau_j, f_\alpha) + E_{com}(\tau_j) \quad (5)$$

The total dynamic energy consumption associated with the execution of all tasks of an application G is determined as:

$$E_d(G) = \sum_{\tau_j=1}^{|V|} E_d(\tau_j) \quad (6)$$

Static energy consumption of the application G is given by:

$$E_s(G) = |U| \times P_s \times SL(G) \quad (7)$$

where $SL(G)$ denotes the schedule length of an application G , generated by a certain scheduling algorithm. The total energy consumption ($E(G)$) associated with the execution of G is given by the sum of both its static and dynamic energy dissipation components, that is:

$$E(G) = E_s(G) + E_d(G) \quad (8)$$

Reducing a processor's frequency causes a decrease in power and energy but elongates task execution times. Although dynamic power P_d decreases as processor frequency is reduced, it may not always direct to lower energy, as task execution times become higher. Normally, a processor has a critical operating frequency f_{cr} at which energy consumption during the task's execution is minimal. This frequency f_{cr} is represented as [14]: $f_{cr} = \sqrt{P_{ind}/(m-1)C_{ef}}$.

IV. THE PROPOSED SCHEDULERS

This section presents a novel scheduling policy called, *Energy-efficient Real-time DAG Scheduler (ERS)*, which constructs a static schedule assuming worst case execution behavior of the application, in order to ensure a minimum guaranteed energy-saving under all possible execution scenarios.

The ERS Algorithm: *ERS* is a static real-time scheduling policy for uniform multiprocessor platforms consisting of three phases, namely (i) *task prioritization*, (ii) *processor allocation*, and (iii) *deadline meeting*. The first two phases of *ERS* are grouped in a function called “*Energy-aware Scheduler for Uniform Multiprocessors (ESUM)*”. The pseudocode of *ESUM* is presented in Algorithm 1. The first phase (*task prioritization*), which consists of lines 1 - 2 of the function *ESUM*, determines the ranks of all the tasks and generates a task priority order. The second phase of *ERS* (which consists of lines 3 - 7 of Algorithm 1) is used to generate a makespan minimizing schedule while executing tasks in the minimum available processor's frequency.

1) *Task Prioritizing Phase:* The priority of each task (say, τ_j) is determined by its rank ($rank[\tau_j]$) using equation 9. The goal of this phase is to sequentially determine the rank of each task, starting from the sink ($rank[\tau_{exit}] = \bar{\omega}_{exit}$) to the source node, and generate a task priority order vector called *taskList*. A higher rank of a task indicates its higher priority value.

$$rank[\tau_j] = \bar{\omega}_j + \max_{\tau_k \in succ(\tau_j)} \{\bar{c}_{j,k} + rank(\tau_k)\} \quad (9)$$

2) *Processor Allocation Phase:* This phase (lines 3 - 7 of *ESUM*) aims to produce a static schedule by sequentially deciding for each task: (i) a *processor*, (ii) an *actual start time*, and (iii) an *actual finish time*, in the order defined by the vector *taskList*, such that the overall schedule *makespan* is minimized while executing tasks in the minimum available processor's frequency ($f_{low} = \max\{f_{cr}, f_{min}\}$). The task $\tau_j (\in taskList)$ to be scheduled next may be allocated to any available processor in the platform. Each possible task-processor allocation is computed using two parameters: (i) *Effective Start Time (EST)* and (ii) *Effective Finish Time (EFT)*, as defined below.

EST $[\tau_j, u_y]$: The *effective start time* $EST[\tau_j, u_y]$ of task τ_j on processor u_y is determined in equation 10. For the entry task τ_{entry} , $EST[\tau_{entry}, u_y] = 0$.

$$EST[\tau_j, u_y] = \begin{cases} 0, & \text{if } \tau_j = \tau_{entry} \\ \max \left\{ avail[y], \max_{\tau_i \in pred(\tau_j)} \{ AFT[\tau_i] + c_{i,j}^{x,y} \} \right\}, & \text{otherwise} \end{cases} \quad (10)$$

where $avail[y]$ is the earliest time at which processor u_y is ready to execute task and $AFT[\tau_i]$ is the *Actual Finish Time* of task τ_i . The inner $\max\{\dots\}$ block in equation 10 returns the time at which all necessary input data required by τ_j from τ_j 's predecessors arrive at u_y .

EFT $[\tau_j, u_y, f_\alpha]$: Given $EST[\tau_j, u_y]$, the *effective finish time* $EFT[\tau_j, u_y, f_\alpha]$ of task τ_j on u_y at frequency f_α is given by:

$$EFT[\tau_j, u_y, f_\alpha] = EST[\tau_j, u_y] + \omega_j \times \frac{f_{max}}{f_\alpha} \quad (11)$$

Now, the task τ_j is actually mapped to a processor u_y for which $EFT[\tau_j, u_y, f_\alpha]$ is minimal. After τ_j is scheduled on u_y , the EST and EFT of τ_j on u_y become $AST[\tau_j]$ (*Actual Start Time*) and $AFT[\tau_j]$ of task τ_j . The *makespan* of the schedule is equal to *actual finish time* of the exit task $AFT[\tau_{exit}]$.

Algorithm 1: *ESUM*(G, U, χ, f_α)

Input: Task graph $G(V, E)$, processor set U , and execution time of each task at frequency f_α

Output: A schedule which minimizes *makespan*

- 1 Compute $rank[\tau_j]$ using equation 9 for each task τ_j by traversing the DAG G starting from τ_{exit} to τ_j ;
 - 2 Sort tasks in non-increasing order of their $rank[\tau_j]$ values and store in *taskList*;
 - 3 **for** *taskList* $\neq 0$ **do**
 - 4 Extract the first task (say, τ_j) from *taskList*;
 - 5 **for each** processor u_y in U **do**
 - 6 Determine $EFT[\tau_j, u_y, f_\alpha]$ using equation 11;
 - 7 Assign τ_j on processor u_y at frequency f_α for which $EFT[\tau_j, u_y, f_\alpha]$ is minimal;
 - 8 **return** [SL, E]; // Obtained schedule length SL and Energy E
-

3) *Deadline Meeting Phase:* In the previous step, *ESUM* generates a schedule using the processor's lowest possible operating frequency f_{low} . In which the overall schedule length is maximum. The objective of the deadline meeting phase is to bring the schedule length within the application deadline with minimal energy. This phase stringent the task execution by progressively increasing the processor frequency (corresponding to the specific task) in a step-by-step fashion. At each step, the strategy attempts to increase the assigned frequency of a certain task on a processor by one level say, from f_{low} to f_{low+1} . The selection of a (task, frequency)-pair $\langle \tau_j, f_\alpha \rangle$ is based on the key value of a parameter called *benefit-to-cost ratio* BCR_j . Symbolically, BCR_j can be expressed as:

$$BCR_j = \frac{E_{new} - E_{cur} + \delta}{SL_{cur} - SL_{new} + \delta} \quad (12)$$

where E_{cur} and SL_{cur} are energy and makespan of the current schedule and E_{new} and SL_{new} are the new energy and

makespan generated by increasing a frequency (from f_{low} to f_{low+1}) of a task τ_j . We consider $\delta = 0.01$ for our experiments.

ERS uses two different heap data structures, namely *minH* and *maxH*, respectively for min and max heap, based on a key BCR_j (refer equation 12). Insertion to the heaps is carried out considering the different possible cases (encountered during the plausible upgradation of the current schedule $\langle SL_{cur}; \text{with energy } E_{cur} \rangle$ to the new schedule $\langle SL_{new}; \text{with energy } E_{new} \rangle$), as given below:

Case 1: $SL_{new} > SL_{cur}$

- 1) It may be noted that the primary objective of the deadline meeting phase is to reduce the makespan of schedule generated in the previous step. Therefore, case 1 is ignored.

Case 2: $SL_{new} = SL_{cur}$

- 1) $E_{new} \geq E_{cur}$
This case is also ignored as it does not target the objective of deadline meeting phase.
- 2) $E_{new} < E_{cur}$ (refer lines 8-9; Algorithm 3)
In this case, there is no change in makespan but energy has decreased. This is the 2nd favorable condition for this phase. In this the key value BCR_j is inserted to *minH*.

Case 3: $SL_{new} < SL_{cur}$

- 1) $E_{new} > E_{cur}$ (refer lines 8-9; Algorithm 3)
In this case, makespan decreases but energy has increased. This is the 3rd favorable condition for this phase. Here, the key value BCR_j is inserted to *minH*.
- 2) $E_{new} = E_{cur}$ (refer lines 6-7; Algorithm 3)
This is the best case to update the schedule. As there is no change in energy but makespan has decreased. Here, the key value BCR_j is inserted to *maxH*.

Algorithm 2: $ERS(G, U, D)$

Input: DAG $G(V, E)$, processors set U , deadline D ;
Global heaps: *minH* and *maxH*

Output: A *safe* schedule having minimal energy

```

1 Compute execution time  $\chi[\tau_j, f_\alpha]$  of each task at  $f_\alpha = f_{low}$ 
  using equation 1;
2  $[SL_{cur}, E(G)] = ESUM(G, U, \chi, f_{low})$ ;
3 for each task  $\tau_j \in V$  do
4    $\text{getRatio}(\tau_j, f_{low}, \chi[\tau_j, f_{low}], SL_{cur}, E(G))$ ;
5 while  $SL_{cur} > D$  do
6   if  $!maxH.empty()$  then
7      $\langle \tau_j, f_\alpha \rangle = maxH.extract()$ ;
8   else if  $!minH.empty()$  then
9      $\langle \tau_j, f_\alpha \rangle = minH.extract()$ ;
10   $f_\alpha = f_\alpha + 0.1$ ;
11  Update  $\chi[\tau_j, f_\alpha]$  using equation 1;
12   $[SL_{cur}, E(G)] = ESUM(G, U, \chi, f_\alpha)$ ;
13   $\text{getRatio}(\tau_j, f_\alpha, \chi[\tau_j, f_\alpha], SL_{cur}, E(G))$ ;
14 if  $SL_{cur} \leq D$  then
15   Report safe schedule with minimal  $E(G)$ ;
16 else
17   Report unsafe schedule;
```

The pseudocode of *ERS* is presented in Algorithm 2. Line 2 calls to the *ESUM*() function (refer to phase 1 and phase 2

of *ERS*) with the lowest possible frequency f_{low} . Lines 3-4 call to *getRatio*() function and insert the $\langle \text{task}, \text{frequency} \rangle$ -pair $\langle \tau_j, f_\alpha \rangle$ to the respective heaps, as discussed in the above cases. Each iteration of the while loop extracts a $\langle \text{task}, \text{frequency} \rangle$ -pair $\langle \tau_j, f_\alpha \rangle$ in lines 5-13. Lines 10-12 call *ESUM*() by increasing the frequency by 0.1. The while loop continues until a *safe* schedule (i.e., makespan $\leq D$) is generated in a certain iteration, with minimal energy $E(G)$, or ends with an *unsafe* schedule.

Algorithm 3: $\text{getRatio}(\tau_j, f_\alpha, \chi[\tau_j, f_\alpha], SL_{cur}, E_{cur})$

Input: $\tau_j, f_\alpha, \chi[\tau_j, f_\alpha], SL_{cur}, E_{cur}$

Global heaps: *minH* and *maxH*

Output: Update *maxH* and *minH* and *minFH*

```

1 if  $f_\alpha < f_{max}$  then
2    $f_\alpha = f_\alpha + 0.1$ ;
3   Update  $\chi[\tau_j, f_\alpha]$  using equation 1 ;
4    $[SL_{new}, E_{new}] = ESUM(G, U, \chi, f_\alpha)$ ;
5   Calculate  $BCR_j$  using equation 12;
6   if  $SL_{new} < SL_{cur}$  and  $E_{new} = E_{cur}$  then
7      $maxH.insert(\tau_j, f_\alpha, BCR_j)$ ;
8   else if  $(SL_{new} = SL_{cur} \text{ and } E_{new} < E_{cur})$  or  $(SL_{new} < SL_{cur} \text{ and } E_{new} > E_{cur})$  then
9      $minH.insert(\tau_j, f_\alpha, BCR_j)$ ;
10   $f_\alpha = f_\alpha - 0.1$ ;
11  Update  $\chi[\tau_j, f_\alpha]$  using equation 1;
```

Example - 1: Fig. 1 shows an example DAG application having 10 nodes and 15 edges with deadline $D = 100$ ms. The power parameters of the processors are: $P_s = 0.03$, $P_{ind} = 0.08$, $m = 2.7$ and $C_{ef} = 1.2$. The tasks are ordered in a vector called *taskList* according to the non-increasing order of their rank values (i.e., $taskList = \langle \tau_1 = 102, \tau_4 = 77, \tau_2 = 70, \tau_3 = 68, \tau_5 = 66, \tau_6 = 54, \tau_9 = 41, \tau_7 = 34, \tau_8 = 26, \tau_{10} = 10 \rangle$). The Gantt chart in Fig. 2d shows the *safe* schedule generated by *ERS* (makespan 100). It can be noted that from Fig. 2, *ERS* ($E(G) = 160.65$ W) outperforms *GSPM* ($E(G) = 198.37$ W), *SSPM* ($E(G) = 197.44$ W), and *PSPM* ($E(G) = 178.37$ W).

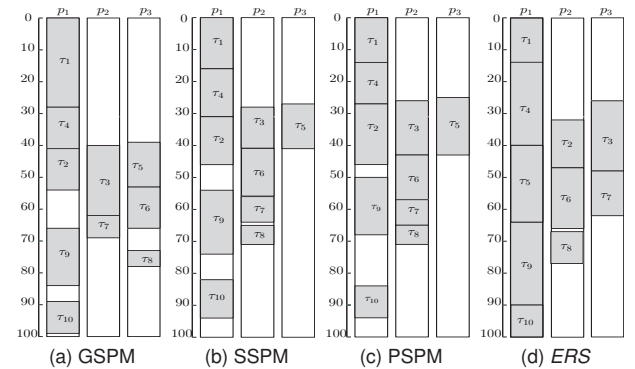


Fig. 2. Gantt charts depicting the schedules: (a) *GSPM* (makespan = 99, $E(G) = 198.37$); (b) *SSPM* (makespan = 94, $E(G) = 197.14$); (c) *PSPM* (makespan = 94, $E(G) = 178.37$); (d) *ERS* (makespan = 100, $E(G) = 160.65$), for the DAG in Fig. 1.

V. EXPERIMENTAL EVALUATIONS

In this section, the performance of the proposed scheduler *ERS* has been assessed through an extensive set of simulation-based experiments. In the following subsections, we explain the experimental setup and the used performance metrics before describing the obtained experimental results.

A. Experimental Setup

The experiments have been conducted using two real-world benchmark DAG applications:

(i) **Gaussian Elimination** [23]: A Gaussian Elimination task graph has $(\nu^2 + \nu - 2)/2$ tasks and $(\nu^2 - \nu - 1)$ edges, where ν is the input parameter called *matrix-size*. For example, Fig. 3a depicts the DAG structure of Gaussian Elimination for *matrix-size* $\nu = 5$. The DAG has 14 nodes and 19 edges.

(ii) **Laplace** [11]: A Laplace task graph has the φ^2 nodes and $2\varphi(\varphi - 1)$ edges, where φ is the input parameter. For example, Fig. 3b depicts a Laplace DAG for $\varphi = 4$. The DAG consists of 16 tasks and 24 edges.

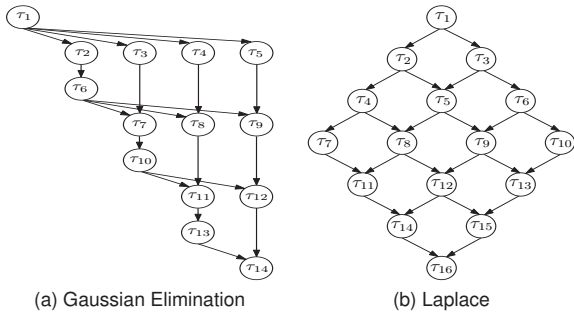


Fig. 3. Benchmark Task Graphs.

Data Generation Framework: Our simulation framework requires the following input parameters: The worst case execution times of tasks are generated randomly with the ranges [10 ms, 100 ms] and data communication between tasks is varied between 10 and 100 Bytes. We have considered platforms of $|U| = \{4, 8, 12, 16, 20, 24\}$ processors for the experiments. The data transfer rate between any two processors is considered as 1, and the communication startup cost of a processor is zero. The power parameters of the processors are varied as follows: static power $P_s = 0.03$, frequency-independent dynamic power $0.03 \leq P_{ind} \leq 0.07$, dynamic exponent power $2.5 \leq m \leq 3.0$ and switching capacitance $0.08 \leq C_{ef} \leq 1.2$. The processors' frequencies are discrete and vary within the range of 0 to 1 with a precision of 0.1 GHz. These parameters reflect the attributes of some high-performance embedded processors, such as *Intel Mobile Pentium III* and *ARM Cortex-A9*.

Deadline Extension Rate: It may be noted that the *ESUM* scheduler discussed as part of this work usually generates the shortest *makespan* schedule when tasks are executed on the maximum available frequency of the processors (f_{max} ; referred to as $ESUM_{max}$). Therefore, while the *makespan* delivered by $ESUM_{max}$ may be small; the corresponding energy consumption rates may be high. Thus, when the deadline is

lower than $ESUM_{max}$'s makespan, *ERS* can rarely produce a *safe* schedule. Further for a given DAG, *safe* schedules with progressively lower energy dissipation should be delivered by *ERS* as the deadline is relaxed more and more concerning $ESUM_{max}$'s makespan. We define a parameter named *deadline extension rate* (∂) as the ratio of the deadline of a DAG 'G' to the *makespan* generated by $ESUM_{max}$ for 'G'. The performance of *ERS* for energy saving maximization has also been evaluated for different values of *deadline extension rate* $\partial = \{1.1, 1.2, 1.3, 1.4, 1.5, 1.6\}$.

B. Performance Metrics

The performance of the proposed methodology has been evaluated using two different parameters:

1. **Normalized Energy Rate (NER):** This metric reflects the normalized energy saving achieved by *ERS* with respect to the maximum possible energy dissipation obtained when all processors are executed at their maximum frequency during execution of the application. Symbolically, *NER* is represented as follows:

$$NER(ERS) = \frac{E^{ESUM_{max}}(G) - E^{ERS}(G)}{E^{ESUM_{max}}(G)} \quad (13)$$

2. **Run-time:** This metric reflects the average run time (in ms) incurred by *ERS* to produce schedules over data generated through a set of fixed parameter values.

C. Experimental Results

This subsection presents detailed experimental results using two benchmarks: Gaussian Elimination [2] and Laplace [11]. In these experiments, each data point is the average over solutions generated with 250 different DAG-related data to a fixed set of parameters.

Experiment-1: This experiment measures the *Normalized Energy Rates* (NERs) of *ERS*, *GSPM*, *SSPM*, and *PSPM*, for varying values of deadline extension rates (∂). Obtained results for both Gaussian Elimination and Laplace are presented in Fig. 4a and Fig. 4b. The parameters $|V|$, $|U|$, and *CCR* have been fixed at 104, 8 and 0.5, respectively. For both Gaussian Elimination and Laplace, average NER increases as the deadline extension rate (∂) increases, indicating that *ERS* can efficiently harness higher available slack to achieve better energy savings.

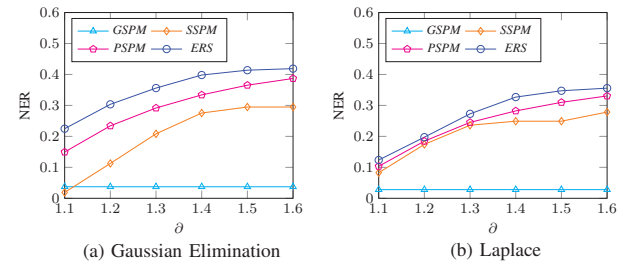


Fig. 4. Normalized Energy Rates w.r.t. varying deadline extension rates.

Experiment-2: This experiment depicts *Normalized Energy rates* (NER) of *ERS* compared to *GSPM*, *SSPM*, and *PSPM*,

for varying values of #tasks ($|V|$), #processors ($|U|$). Fig. 5 shows the results obtained for the Gaussian Elimination task graphs. The results for the Laplace application have not been plotted here, as the trends of the results are similar to that of Gaussian Elimination.

Fig. 5a depicts the obtained NER values for different number of tasks ($|V|$) vary between 25 and 200. Value of the parameters $|U|$, CCR and ∂ are set to 8, 0.5 and 1.2, respectively. It is observed that *ERS* delivers better results than *GSPM*, *SSPM*, and *PSPM* in terms of achieved NER values in all cases.

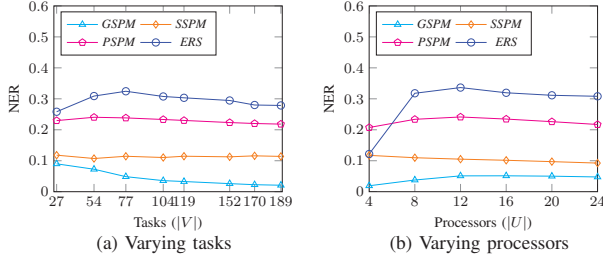


Fig. 5. Normalized Energy Rates for Gaussian Elimination.

Fig. 5b shows the NER results as the number of processors varies from 4 to 24. The parameters $|V|$, CCR and ∂ are set to 104, 0.5 and 1.2, respectively. As an example of *ERS*'s performance, it may be observed in Fig. 5b (Gaussian Elimination) that for $|U| = 12$, the average energy savings of *ERS* is higher than *GSPM*, *SSPM*, and *PSPM* by approximately 84%, 68%, and 28%, respectively.

Experiment-3: This experiment reflects the *run-times* of *ERS*, *GSPM*, *SSPM*, and *PSPM*, for varying values of #tasks ($|V|$) (refer Table I) using Gaussian Elimination. It is observed that the *run-times* of all the algorithms strictly increase with the number of tasks.

TABLE I
AVERAGE RUN-TIMES (IN ms) OF DIFFERENT ALGORITHMS

$ V $	<i>ERS</i>	<i>GSPM</i>	<i>SSPM</i>	<i>PSPM</i>
27	63.83	23.28	3.48	0.03
54	116.55	36.85	3.90	0.15
77	183.05	42.87	4.33	0.62
104	349.03	52.35	5.47	0.50
119	505.61	60.52	5.90	0.50

VI. CONCLUSION

In this article, we have proposed an efficient list-based heuristic strategy called *Energy-efficient Real-time DAG Scheduler (ERS)* for energy-aware real-time scheduling of DAG-structured applications to be executed on distributed systems. *ERS* effectively selects the appropriate processing frequency for each task-to-processor pair in the system to maximize overall energy saving while satisfying constraints related to resource, deadline and task precedence. We have evaluated the performance of the proposed strategy using two benchmark applications. Obtained experimental results show that *ERS* is

able to deliver better performance in terms of energy savings than state-of-the-art works such as *GSPM*, *SSPM*, and *PSPM*.

REFERENCES

- [1] G. C. Buttazzo, *Hard real-time computing systems*. Springer Science & Business Media, 2011, vol. 24.
- [2] D. Senapati, A. Sarkar, and C. Karfa, "Presto: A penalty-aware real-time scheduler for task graphs on heterogeneous platforms," *IEEE Transactions on Computers*, vol. 71, no. 2, pp. 421–435, 2022.
- [3] H. Topcuoglu, S. Hariri, and M.-y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE TPDS*, vol. 13, no. 3, pp. 260–274, 2002.
- [4] E. G. Coffman and J. L. Bruno, *Computer and job-shop scheduling theory*. John Wiley & Sons, 1976.
- [5] J. D. Ullman, "Np-complete scheduling problems," *Journal of Computer and System sciences*, vol. 10, no. 3, pp. 384–393, 1975.
- [6] D. Senapati, A. Sarkar, and C. Karfa, "Performance-effective dag scheduling for heterogeneous distributed systems," ser. ICDN '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 234–235.
- [7] Y.-K. Kwok and I. Ahmad, "Dynamic critical-path scheduling: An effective technique for allocating task graphs to multiprocessors," *IEEE TPDS*, vol. 7, no. 5, pp. 506–521, 1996.
- [8] G. C. Sih and E. A. Lee, "A compile-time scheduling heuristic for interconnection-constrained heterogeneous processor architectures," *IEEE TPDS*, vol. 4, no. 2, pp. 175–187, 1993.
- [9] H. El-Rewini and T. G. Lewis, "Scheduling parallel pro. tasks onto arbitrary target machines," *JPDC*, vol. 9, no. 2, pp. 138–153, 1990.
- [10] D. Senapati, A. Sarkar, and C. Karfa, "Hmds: A makespan minimizing dag scheduler for heterogeneous distributed systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, no. 5s, Sep. 2021. [Online]. Available: <https://doi.org/10.1145/3477037>
- [11] D. Senapati, K. Rajesh, C. Karfa, and A. Sarkar, "Tmds: Temperature-aware makespan minimizing dag scheduler for heterogeneous distributed systems," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 28, no. 6, oct 2023. [Online]. Available: <https://doi.org/10.1145/3616869>
- [12] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE TPDS*, vol. 25, no. 3, pp. 682–694, 2014.
- [13] R. Mishra, N. Rastogi, D. Zhu, D. Mossé, and R. Melhem, "Energy aware scheduling for distributed real-time systems," in *Proceedings International Parallel and Distributed Processing Symposium*. IEEE, 2003, pp. 9–pp.
- [14] D. Senapati, A. Sarkar, and C. Karfa, "Energy-aware real-time scheduling of multiple periodic dags on heterogeneous systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 8, pp. 2447–2460, 2023.
- [15] M. Bambagini, M. Marinoni, H. Aydin, and G. Buttazzo, "Energy-aware scheduling for real-time systems: A survey," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 15, no. 1, pp. 1–34, 2016.
- [16] M. Weiser, B. Welch, A. Demers, and S. Shenker, "Scheduling for reduced cpu energy," *Mobile Computing*, pp. 449–471, 1994.
- [17] F. Yao, A. Demers, and S. Shenker, "A scheduling model for reduced cpu energy," in *Proceedings of IEEE 36th annual foundations of computer science*. IEEE, 1995, pp. 374–382.
- [18] H.-S. Yun and J. Kim, "On energy-optimal voltage scheduling for fixed-priority hard real-time systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 2, no. 3, pp. 393–430, 2003.
- [19] M. Li, B. J. Liu, and F. F. Yao, "Min-energy voltage allocation for tree-structured tasks," *Journal of Combinatorial Optimization*, vol. 11, no. 3, pp. 305–319, 2006.
- [20] R. Jejurikar and R. Gupta, "Energy-aware task scheduling with task synchronization for embedded real-time systems," *IEEE TCAD*, vol. 25, no. 6, pp. 1024–1037, 2006.
- [21] V. Devadas and H. Aydin, "Coordinated power management of periodic real-time tasks on chip multiprocessors," in *International Conference on Green Computing*. IEEE, 2010, pp. 61–72.
- [22] G. Chen, K. Huang, and A. Knoll, "Energy optimization for real-time multiprocessor system-on-chip with optimal dvfs and dpm combination," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 13, no. 3s, pp. 1–21, 2014.
- [23] S. K. Roy, R. Devaraj, A. Sarkar, and D. Senapati, "Slaqa: Quality-level aware scheduling of task graphs on heterogeneous distributed systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, no. 5, pp. 1–31, 2021.